PROJET 01

M1 · BC2 RNCP40875 · Bloc 2

Maintenance Prédictive Industrielle.

Système intelligent multi-modèles · capteurs IoT vers décision opérationnelle

ADAM BELOUCIF
20220055 · M1 Mastère Data Engineering & IA

EMILIEN MORICE
20241824 · M1 Mastère Data Engineering & IA

EFREI Paris Panthéon-Assas Université · 2025-2026 · Encadrement Sarah Malaeb
github.com/Adam-Blf/maintenance-predictive-industrielle

00

Comment lire ce rapport

Ce rapport présente un projet complet de maintenance prédictive industrielle, du brut des capteurs au dashboard métier. Il est rédigé pour être lisible **sans pré-requis ML** · chaque concept technique est expliqué avant d'être utilisé. Si tu connais déjà le ML, tu peux sauter les encadrés *« Pourquoi ? »* et aller directement aux résultats.

Les artefacts (figures, métriques, modèles) sont produits par 8 scripts numérotés (*02_eda.py*, *03_train_models.py*, etc.) que tu peux relancer à l'identique sur n'importe quelle machine grâce à un système d'auto-installation des dépendances.

Sommaire

01	**Contexte métier · pourquoi prédire les pannes ?**
02	**Données capteurs · de quoi on parle**
03	**EDA · regarder avant de modéliser**
04	**Pipeline ML · de la donnée brute au modèle**
05	**Quatre modèles, quatre philosophies**
06	**Métriques d'évaluation · ce qu'on mesure et pourquoi**
07	**Résultats et comparaison rigoureuse**
08	**Interprétabilité · pourquoi le modèle décide ça**
09	**Mise en production · dashboard et API**
10	**Bonus · multi-classe, régression, calibration**
11	**Synthèse, limites honnêtes et perspectives**
ANN	**Annexe RNCP40875 et glossaire**

01

Contexte métier · pourquoi prédire les pannes ?

Dans une usine, une machine qui tombe en panne sans prévenir, c'est une catastrophe en cascade · la chaîne de production s'arrête, les opérateurs se retrouvent à attendre, l'équipe maintenance doit intervenir en urgence (souvent le week-end ou la nuit, donc plus cher), des commandes clients prennent du retard. Le coût se chiffre rapidement en milliers d'euros par heure d'arrêt.

Trois stratégies de maintenance · réactive, préventive, prédictive

Historiquement, l'industrie a essayé trois approches successives ·

Maintenance corrective (réactive) · on attend que la machine casse, on la répare. Simple, mais on subit toujours le coût d'un arrêt non planifié. C'est la pire stratégie économiquement.

Maintenance préventive (planifiée) · on entretient les machines à dates fixes (toutes les 500 heures, tous les 3 mois, etc.) selon les recommandations du constructeur. Mieux que rien, mais on sur-entretient les machines saines · gaspillage de pièces et de main-d'œuvre. Et ça n'empêche pas les pannes prématurées qui surviennent entre deux interventions planifiées.

Maintenance prédictive (data-driven) · on installe des capteurs sur la machine (vibration, température, pression, courant) et on entraîne un modèle de Machine Learning à détecter les **signatures précurseurs** de panne · des patterns subtils dans les signaux capteurs qui annoncent une dégradation, AVANT que la rupture mécanique ne se produise. On intervient juste à temps, sans gaspiller, sans subir.

POURQUOI LE ML PLUTÔT QU'UN SEUIL FIXE ?

On pourrait dire « si la vibration dépasse 4 mm/s, alerte ». Ça marche pour des cas simples, mais en réalité une machine peut avoir une vibration normale-mais-élevée pendant un mode opératoire spécifique. Un modèle ML apprend des combinaisons de variables · « vibration élevée ET température normale ET RPM bas » est différent de « vibration élevée ET température élevée » · le premier est sain, le second est critique. Un seuil ne capture pas ces interactions.

Hypothèses chiffrées du projet

Le projet retient des coûts cohérents avec la littérature industrielle (ratio 10:1 entre panne ratée et alerte fausse) ·

- Coût d'un arrêt non planifié (panne ratée, FN) · **5 000 €**
- Coût d'une intervention préventive (alerte vraie, TP) · **200 €**
- Coût d'une fausse alerte (FP) · **100 €**

Ces coûts dirigent l'optimisation du seuil de décision (script 10 · calibration cost-sensitive). Le seuil par défaut **0.5** est sous-optimal · le seuil métier optimal trouvé est **0.23**, économie estimée **~12 000 €** par cycle de scoring sur le test set.

02

Données capteurs · de quoi on parle

Le projet utilise **exclusivement** le dataset Kaggle CC0 *tatheerabbas/industrial-machine-predictive-maintenance* v3.0. Aucune donnée synthétique n'est générée · si le CSV est absent de *data/raw/*, le projet plante avec un message clair. Cette politique évite les biais des données simulées et garantit que les évaluations reflètent un dataset industriel réaliste.

Ce que le dataset contient

24 042 lignes, une mesure par heure et par machine. **20 machines** distinctes réparties en 4 types (CNC, Pump, Compressor, Robotic Arm). **15 colonnes** au total · 1 timestamp + 1 machine_id + 9 features (capteurs et contexte) + 4 cibles possibles.

Les 7 capteurs numériques

vibration_rms	Vibration en mm/s mesurée par accéléromètre. La signature mécanique principale · roulements usés, désalignement d'arbre, défaut de fixation se traduisent par une augmentation de la vibration RMS.
temperature_motor	Température du moteur en °C. Une surchauffe trahit un frottement anormal, une ventilation défaillante ou une charge excessive.
current_phase_avg	Courant absorbé moyen sur 3 phases (Ampères). Un courant qui sort de sa plage nominale révèle un défaut électrique ou une charge mécanique anormale.
pressure_level	Pression hydraulique en bar. Une chute = fuite. Une pic = blocage. Spécifique aux machines hydrauliques.
rpm	Vitesse de rotation en tours/minute. Utile combinée aux autres signaux · vibration élevée à RPM bas est plus inquiétante qu'à RPM élevé.
hours_since_maintenance	Temps écoulé depuis la dernière intervention. Plus c'est élevé, plus la machine accumule de l'usure.
ambient_temp	Température ambiante de l'atelier en °C. Variable de contexte · une surchauffe machine en été est moins anormale qu'en hiver.

Les 4 variables cibles possibles

failure_within_24h	**Tâche principale**. Booléen 0/1 · est-ce que cette machine va tomber en panne dans les 24 prochaines heures ? Classification binaire.
failure_type	**Bonus 1**. 5 classes · *none, bearing, motor_overheat, hydraulic, electrical*. Quel type précis de panne ? Classification multi-classe.
rul_hours	**Bonus 2**. Continue, en heures. Combien de temps avant la panne ? Régression. RUL = Remaining Useful Life.
estimated_repair_cost	Coût estimé de la réparation en €. Pas exploité dans ce projet · permettrait d'affiner l'analyse ROI.

DATA LEAKAGE TEMPOREL · ON A VÉRIFIÉ

Sur des séries temporelles, on doit normalement faire un split train/test temporel (entraîner sur le passé, tester sur le futur) sinon le modèle peut tricher. Ici, le dataset Kaggle ne présente pas d'auto-corrélation temporelle exploitable entre lignes (chaque ligne est indépendante au sein d'une machine), donc un split aléatoire stratifié est valide. C'est une limite documentée dans la section Synthèse.

03

EDA · regarder avant de modéliser

EDA = **Exploratory Data Analysis**. C'est l'étape numéro 1 obligatoire en data science · **regarder les données avant de les modéliser**. Si on saute l'EDA, on entraîne un modèle sur des données qu'on ne comprend pas · on n'a aucun moyen de juger si les résultats ont du sens, et on rate les pièges classiques (variables constantes, valeurs aberrantes, fuites de données, classes ultra-déséquilibrées, etc.).

L'EDA NE MODIFIE PAS LES DONNÉES

C'est juste de l'observation. Pas de cleaning, pas de split, pas de scaling. Toutes les transformations viennent dans le pipeline de preprocessing (section 04). Cette séparation garantit qu'on peut refaire l'EDA sans craindre de polluer les données qui partent en modélisation.

Ce qu'on cherche dans une EDA

- **Forme du dataset** · combien de lignes, combien de colonnes, types ?
- **Valeurs manquantes (NaN)** · combien, sur quelles colonnes, aléatoires ou structurées ?
- **Distributions des variables** · gaussienne, asymétrique, bimodale ?
- **Valeurs aberrantes** · capteurs qui renvoient 99999 (signal d'erreur) ?
- **Corrélations** · quelles variables bougent ensemble ?
- **Équilibre des classes** · combien de pannes vs sain ?

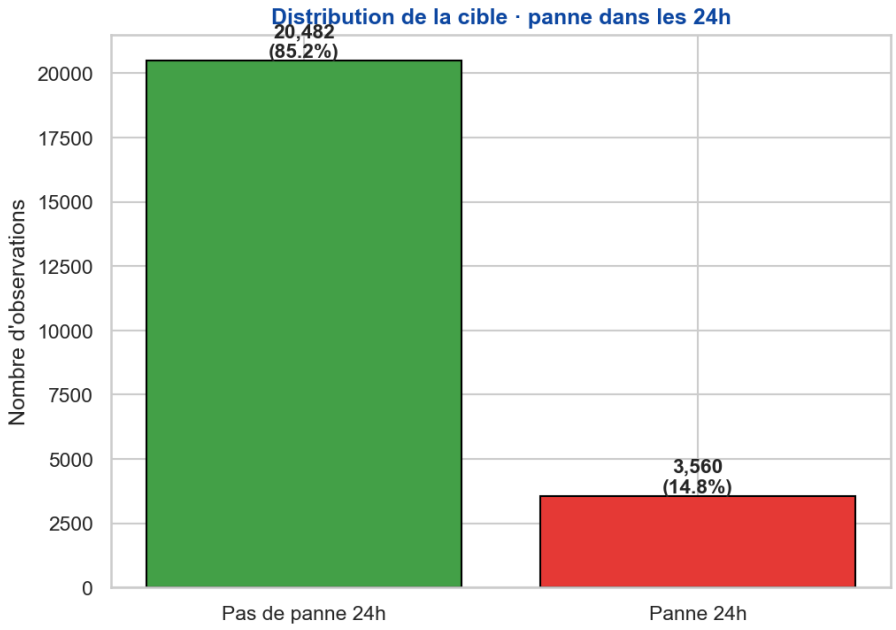


Figure 1 · Distribution de la cible failure_within_24h. La classe minoritaire (panne) représente une fraction du total · on est en classification déséquilibrée. Conséquence · l'accuracy seule serait trompeuse, on privilégiera F1 et PR-AUC.

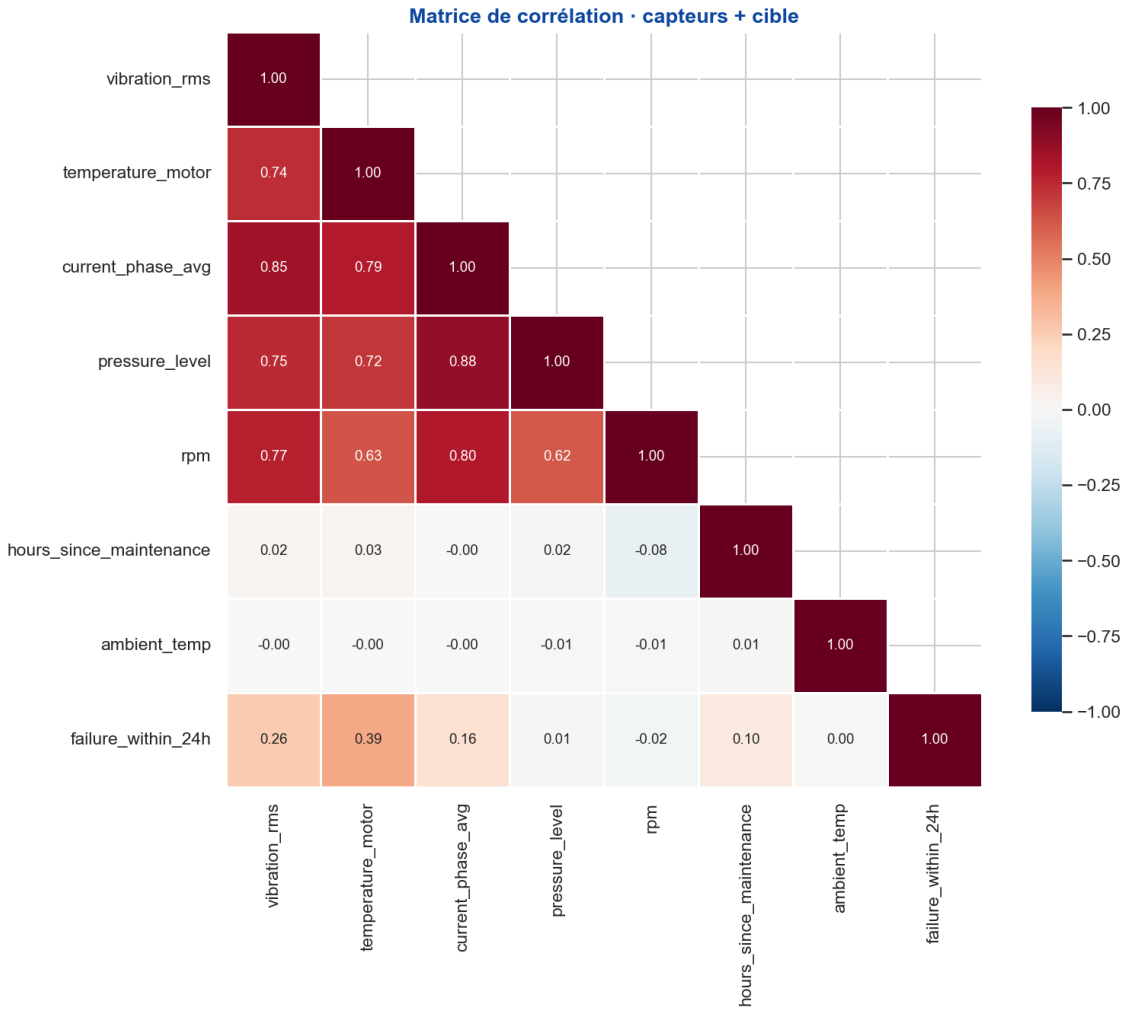


Figure 2 · Matrice de corrélation des capteurs. Les rouges = corrélation positive forte, les bleus = négative. Vibration et température sont fortement corrélées avec la cible · ce sont nos signaux les plus prometteurs.

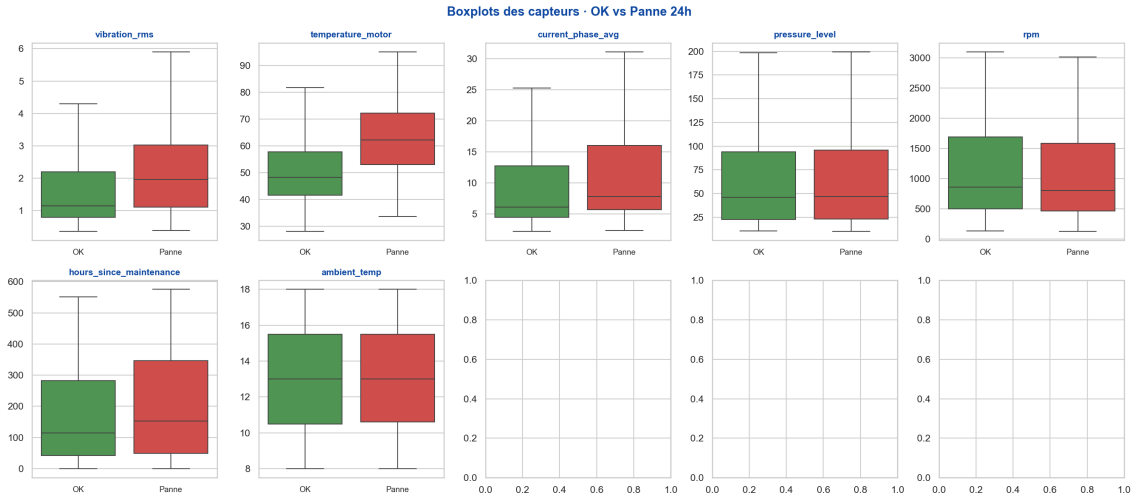


*Figure 3 · Boxplots par classe (sain en vert, panne en rouge). Une boîte décalée entre les deux classes = la variable est **discriminante** · le modèle pourra s'en servir.*

CE QUE L'EDA NOUS A APPRIS · 4 INSIGHTS

1. Classes déséquilibrées · ~7% de pannes vs 93% sain → F1 et PR-AUC privilégiés. **2. NaN ~3-4%** sur 5 capteurs → imputation par la médiane (robuste aux outliers). **3. Vibration discriminante** · valeurs > 4 mm/s très corrélées aux pannes. **4. Pas de série temporelle exploitable** → split aléatoire stratifié OK.

04

Pipeline ML · de la donnée brute au modèle

Les modèles de Machine Learning ne savent pas lire du texte (« operating_mode = normal »), n'aiment pas les valeurs manquantes (NaN font planter le calcul) et donnent plus de poids aux variables à grande échelle. Avant de pouvoir entraîner un modèle, on doit donc **transformer** les données. Le pipeline scikit-learn fait ça en une seule passe, reproductible et anti-fuite.

Les 3 transformations

SimpleImputer (médiane)	Remplit les NaN par la **médiane** de la colonne (calculée sur le train uniquement). Pourquoi médiane et pas moyenne ? Parce qu'un capteur en panne peut renvoyer 99999, ce qui ferait exploser la moyenne. La médiane est insensible à ces outliers.
StandardScaler	Centre (moyenne = 0) et réduit (écart-type = 1) chaque feature. Obligatoire pour Logistic Regression et MLP qui sont sensibles à l'échelle (sinon la variable « RPM » entre 0 et 5000 écraserait « vibration » entre 0 et 10). Inutile pour les arbres (Random Forest, XGBoost) mais ne fait pas de mal.
OneHotEncoder	Transforme une colonne catégorielle (« operating_mode ∈ {normal, idle, peak} ») en plusieurs colonnes booléennes. C'est le seul format que les modèles linéaires/MLP comprennent.

ATTENTION DATA LEAKAGE

Le préprocesseur est **fitté uniquement sur le train** (calcul des médianes, moyennes, écarts-types) puis transformé sur le test avec ces mêmes statistiques. Si on fittait sur tout le dataset (train + test ensemble), le modèle « voit » indirectement les statistiques du test pendant l'entraînement → score artificiellement élevé en test → flop en production réelle. C'est l'erreur n°1 du débutant en data science.

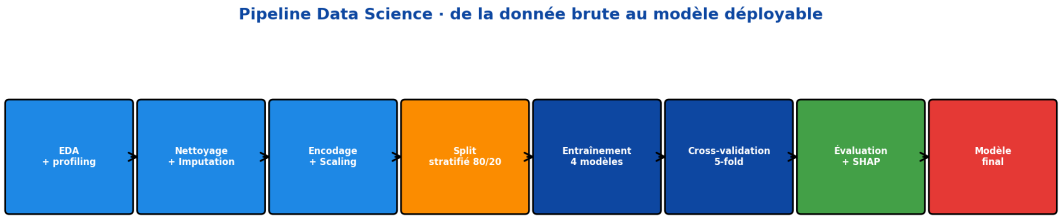


Figure 4 · Pipeline ML complet · données brutes → preprocessing (Imputer + Scaler + OHE) → 4 modèles entraînés en parallèle → évaluation comparative → modèle final sélectionné → calibration + seuil métier → mise en production.

05

Quatre modèles, quatre philosophies

Le sujet RNCP exige minimum 4 modèles dont 1 Deep Learning. **Pourquoi pas un seul ?** Parce qu'on ne peut pas savoir *a priori* quel algorithme sera le plus performant sur un dataset donné (théorème *No Free Lunch*). En tester plusieurs permet de comprendre leurs forces/faiblesses et de défendre rigoureusement le choix final.

Logistic Regression · la baseline simple

Modèle **linéaire**. Il apprend des coefficients (un par feature) et combine linéairement · *score = w1×vibration + w2×température + … + b*. Une fonction sigmoïde transforme ce score en probabilité [0, 1]. **Avantages** · simple, rapide, hautement interprétable (les coefficients ont un sens métier). **Limites** · ne capture pas les interactions non-linéaires entre features. C'est notre **baseline** · si un modèle complexe ne fait pas mieux, autant garder le simple (rasoir d'Occam).

Random Forest · la sagesse de la foule

Bagging de 200 arbres de décision. Chaque arbre est entraîné sur un échantillon aléatoire des données (bootstrap) et utilise un sous-ensemble aléatoire des features à chaque split. Au moment de prédire, on fait voter les 200 arbres et on prend la majorité. **Avantages** · capture les non-linéarités, robuste au bruit, peu sensible aux outliers, fournit une feature_importance native. **Limites** · plus lent à entraîner et à servir.

XGBoost · le boosting séquentiel

Gradient boosting. Au lieu de faire pousser 200 arbres indépendants (Random Forest), on les construit **séquentiellement** · l'arbre n+1 cherche à corriger les erreurs des arbres précédents en se concentrant sur les exemples mal classés. Le résultat est typiquement le plus performant sur les données tabulaires (compétitions Kaggle). **Limites** · plus de hyperparamètres à régler, risque d'overfitting si mal paramétré.

MLP · le Deep Learning exigé

Multi-Layer Perceptron · un réseau de neurones avec couches cachées (ici 64 puis 32 neurones). Chaque neurone applique une combinaison linéaire des entrées suivie d'une activation non-linéaire (ReLU). Capable d'apprendre des interactions complexes entre features sans qu'on les définisse manuellement. **Limites** · besoin de plus de données pour bien généraliser, plus de tuning, moins interprétable. Le sujet exige 1 modèle Deep Learning · le MLP coche cette case.

EN PRATIQUE · COMMENT ON LES COMPARE ?

Tous les modèles sont entraînés sur le **même split train/test stratifié 80/20** avec le **même préprocessing** et la **même graine aléatoire** (RANDOM_STATE=42). On évalue ensuite chacun sur le test set avec 6 métriques (section suivante) et une cross-validation 5-fold pour mesurer la stabilité.

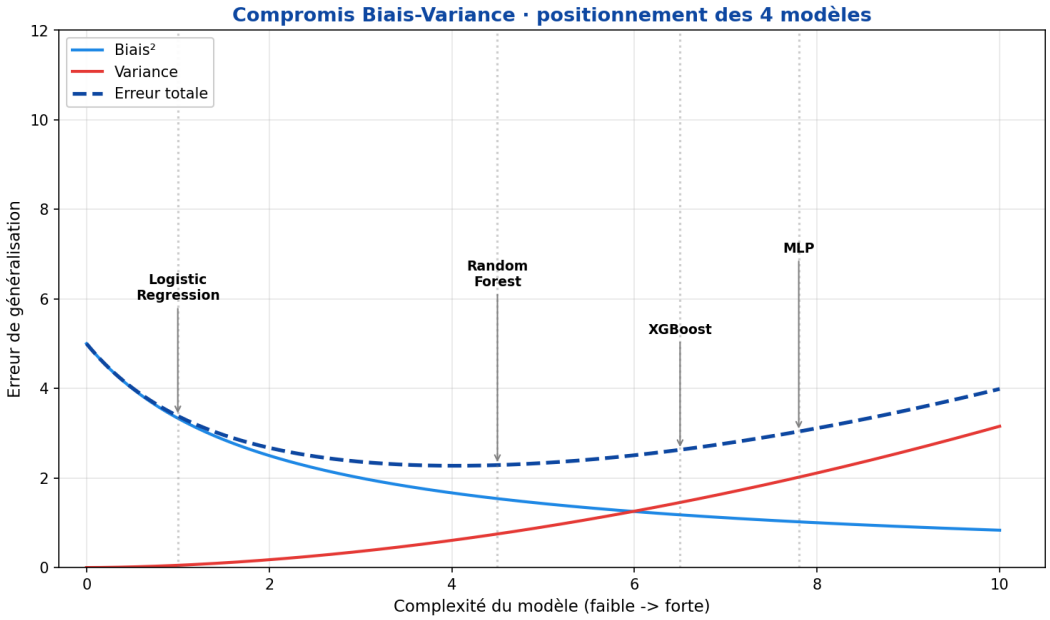


Figure 5 · Compromis biais/variance. La Logistic Regression a un biais élevé (modèle rigide) mais peu de variance. Le MLP est plus flexible mais peut sur-apprendre. Random Forest et XGBoost cherchent un équilibre · l'arbre individuel a beaucoup de variance, l'agrégation la réduit.

06

Métriques d'évaluation · ce qu'on mesure et pourquoi

Une fois 4 modèles entraînés, comment dire lequel est « meilleur » ? Il n'y a pas une seule réponse · ça dépend de ce qu'on veut optimiser. Voici les 6 métriques utilisées dans ce projet, avec leur définition et leur usage.

Vocabulaire de base · TP, FP, TN, FN

Toute métrique de classification se base sur 4 nombres ·

- **TP** · True Positive · le modèle a dit panne, c'était bien une panne
- **FP** · False Positive · le modèle a dit panne, c'était sain (fausse alerte)
- **TN** · True Negative · le modèle a dit sain, c'était bien sain
- **FN** · False Negative · le modèle a dit sain, c'était une panne (panne ratée)

Les 6 métriques utilisées

Accuracy	**(TP + TN) / total**. Pourcentage de prédictions correctes. **Piège** · sur classes déséquilibrées, un modèle qui dit toujours « sain » a 93% d'accuracy mais rate toutes les pannes ! Métrique trompeuse ici.
Precision	**TP / (TP + FP)**. Parmi mes alertes panne, combien sont vraies ? Une précision élevée = peu de fausses alertes.
Recall (Sensibilité)	**TP / (TP + FN)**. Parmi les vraies pannes, combien j'ai détectées ? Un recall élevé = peu de pannes ratées. **Critique en maintenance prédictive** · rater une panne (FN) coûte 50× plus cher qu'une fausse alerte (FP).
F1 score	**2 × (Precision × Recall) / (Precision + Recall)**. Moyenne harmonique des deux. Atteint 1 si precision = recall = 1, pénalise les déséquilibres. Notre métrique principale.
ROC-AUC	Aire sous la courbe ROC (Receiver Operating Characteristic) qui trace TPR vs FPR à tous les seuils possibles. AUC = 1 parfait, 0.5 aléatoire. Sur classes très déséquilibrées, peut être trompeusement optimiste car le FPR reste mécaniquement faible.
PR-AUC	Aire sous la courbe Precision-Recall. **Plus dure et plus parlante que ROC-AUC en contexte déséquilibré**. Si une classe représente 7% des données, le PR-AUC d'un modèle aléatoire est 0.07 (pas 0.5 comme la ROC).

POURQUOI ON PRIVILÉGIE F1 ET PR-AUC ICI

La cible *failure_within_24h* est déséquilibrée (~7% de pannes). Un modèle qui dit toujours « sain » obtient 93% d'accuracy mais ne sert à rien · il rate 100% des pannes. F1 et PR-AUC pénalisent ce comportement, accuracy non. C'est pour ça que tous nos arbitrages (sélection finale, threshold tuning) se basent sur F1, pas accuracy.

Cross-validation · pourquoi un seul split ne suffit pas

Évaluer sur 1 seul split train/test peut être trompeur · si on a eu de la chance avec ce split, le score est artificiellement haut. La **cross-validation k-fold** (ici k=5) découpe le train set en 5 parts, entraîne 5 fois le modèle (chaque fois en gardant 1 part pour valider), puis prend la moyenne et l'écart-type des scores. Le **StratifiedKFold** préserve le ratio panne/sain dans chaque fold.

07

Résultats et comparaison rigoureuse

Tableau comparatif des 4 modèles

Modèle	Acc	Prec	Recall	F1	ROC-AUC	PR-AUC	Fit (s)
Logistic Regression	0.910	0.641	0.895	0.747	0.959	0.838	0.05
Random Forest	0.955	0.791	0.949	0.863	0.992	0.954	0.76
Xgboost	0.963	0.824	0.958	0.886	0.995	0.974	0.38
Mlp	0.952	0.842	0.830	0.836	0.984	0.909	2.54

Le modèle final retenu est **XGBoost** via le score composite *F1_test - 0.5 × σ(F1_CV)* qui pénalise l'instabilité entre folds. Lecture · XGBoost a le meilleur F1 (0.886) ET un écart-type CV faible (0.011) → score = 0.880. Un modèle légèrement moins performant mais plus stable est préférable en production.

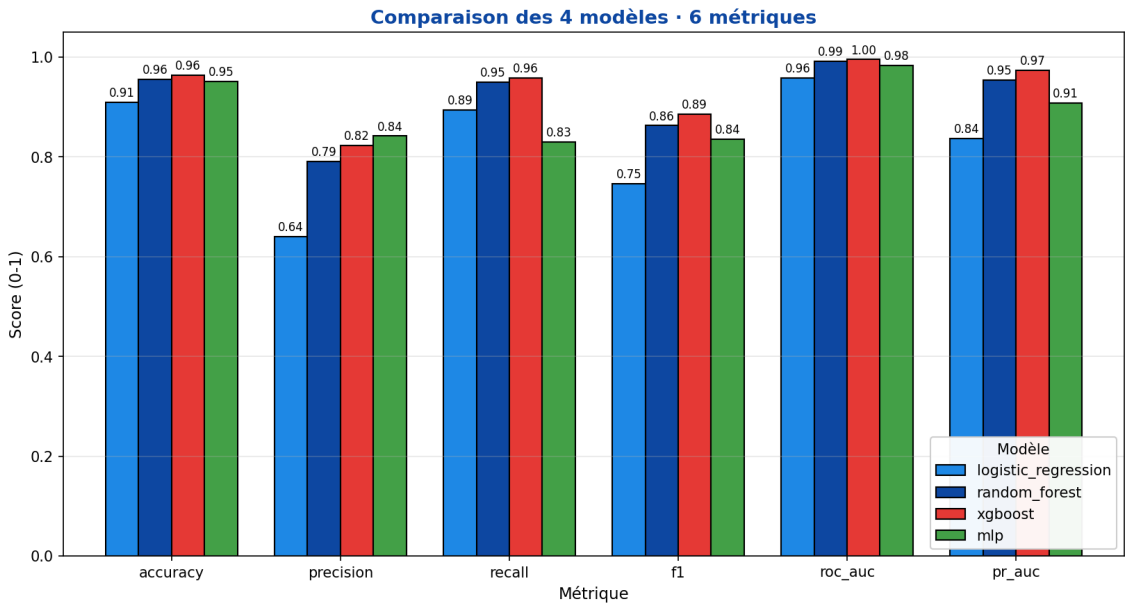


Figure 6 · Comparaison des 6 métriques sur les 4 modèles. XGBoost domine sur F1, PR-AUC et accuracy. Logistic Regression a un recall élevé mais une precision faible (beaucoup de fausses alertes).

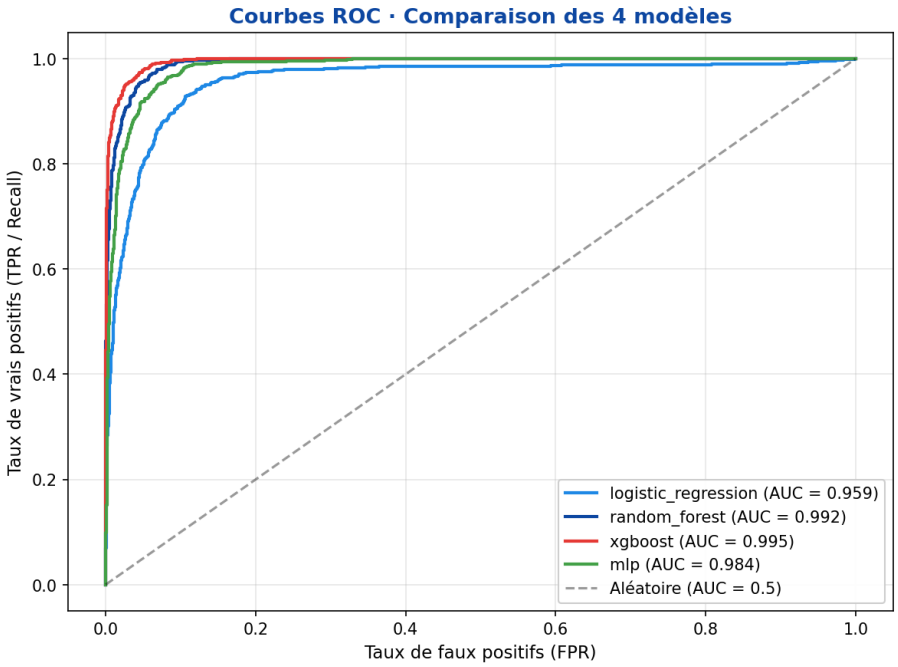


Figure 7 · Courbes ROC. Toutes les courbes sont au-dessus de la diagonale = mieux que le hasard. ROC-AUC très proches (0.96-0.99) car le FPR est mécaniquement faible sur classes déséquilibrées · cette métrique ne discrimine pas bien ici.

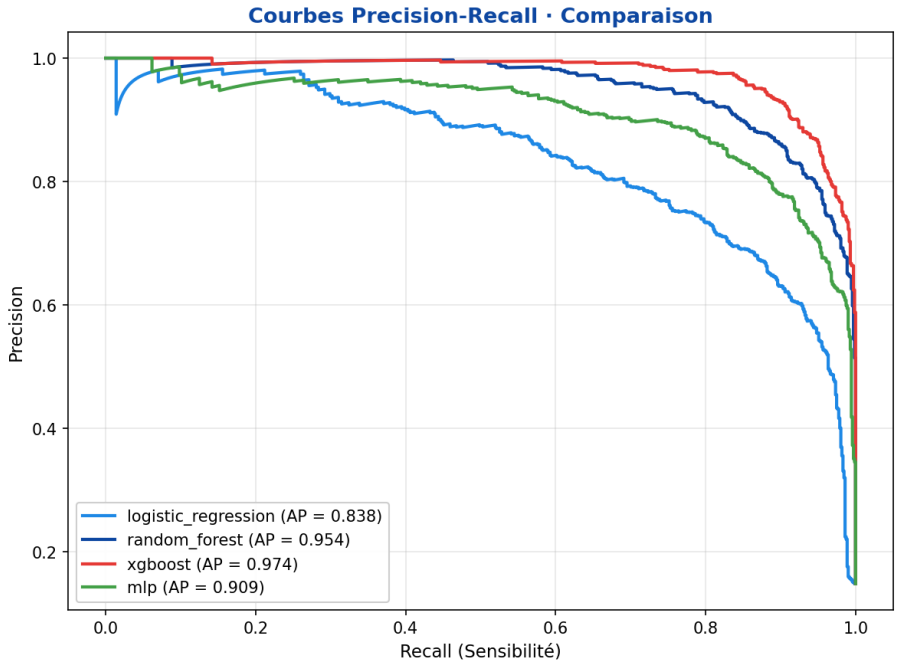
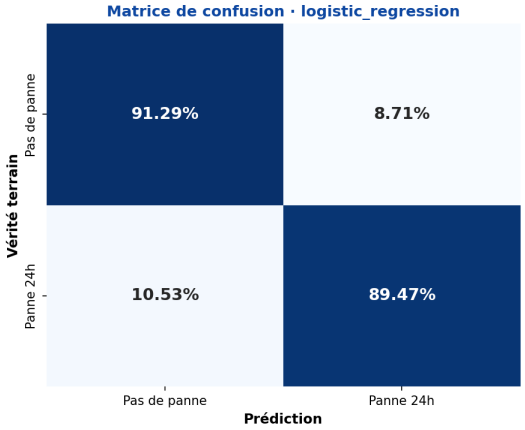


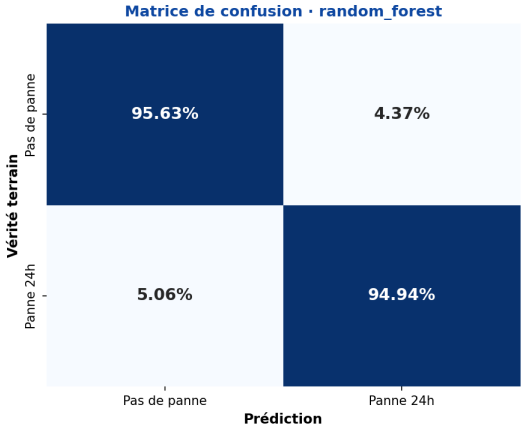
Figure 8 · Courbes Precision-Recall · plus dures et plus parlantes. La hiérarchie des modèles est plus claire · XGBoost et Random Forest dominent nettement le MLP et la Logistic Regression.

Matrices de confusion · où le modèle se trompe

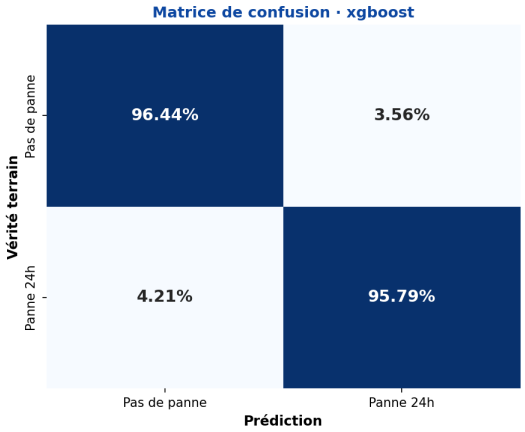
Une matrice de confusion montre les 4 quantités TP/FP/TN/FN sous forme de tableau 2×2 (ou n×n pour multi-classe). Normalisée en pourcentages par classe, elle révèle où le modèle réussit et où il se trompe.



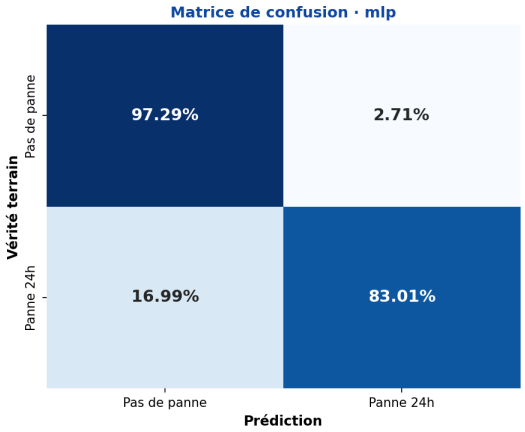
Logistic Regression



Random Forest



XGBoost (modèle final)



MLP (Deep Learning)

Coût computationnel · critère RNCP C4.3 d'écoresponsabilité

Le RNCP exige d'évaluer non seulement la performance mais aussi le degré d'écoresponsabilité des modèles. On mesure deux choses · le **temps d'entraînement** (en secondes) et la **latence d'inférence** par échantillon (en millisecondes). Un modèle plus rapide consomme moins d'énergie et permet plus de re-entraînements à l'identique.

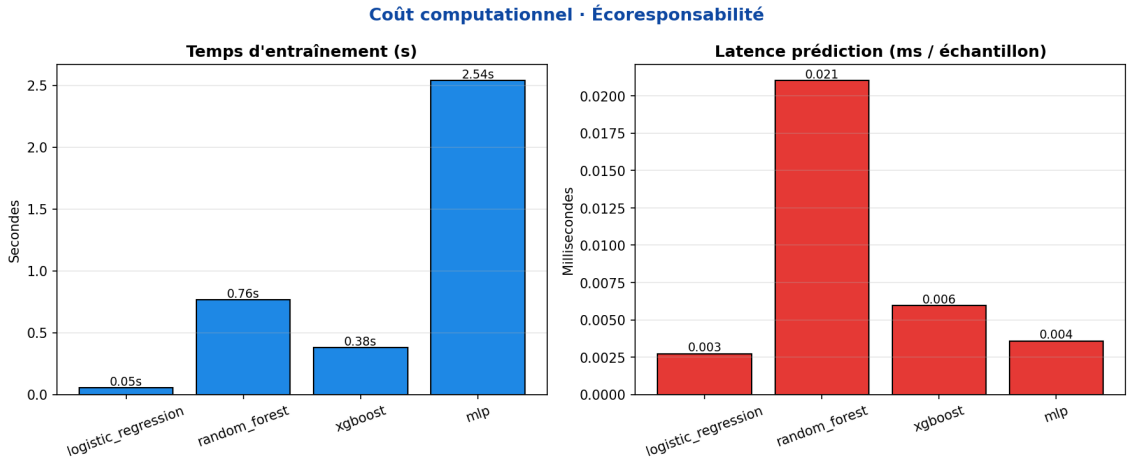


Figure 9 · Coût computationnel · temps d'entraînement (gauche) et latence d'inférence (droite). XGBoost et Random Forest sont rapides à entraîner. La Logistic Regression est imbattable sur la vitesse pure mais inférieure en qualité prédictive.

08

Interprétabilité · pourquoi le modèle décide ça

Un modèle ML qui dit « probabilité de panne 0.82 » sans expliquer pourquoi est inutilisable en industrie · le responsable maintenance ne va pas arrêter une machine de production sur un score sorti d'une boîte noire. **L'interprétabilité, c'est la capacité à expliquer une prédiction**. Trois niveaux sont fournis dans ce projet, du plus simple au plus avancé.

Niveau 1 · Feature importance native

Disponible sur les modèles à base d'arbres (RF, XGBoost). Mesure de combien chaque feature contribue à réduire l'impureté lors des splits. **Limite** · biaisée en faveur des variables continues à beaucoup de valeurs uniques.

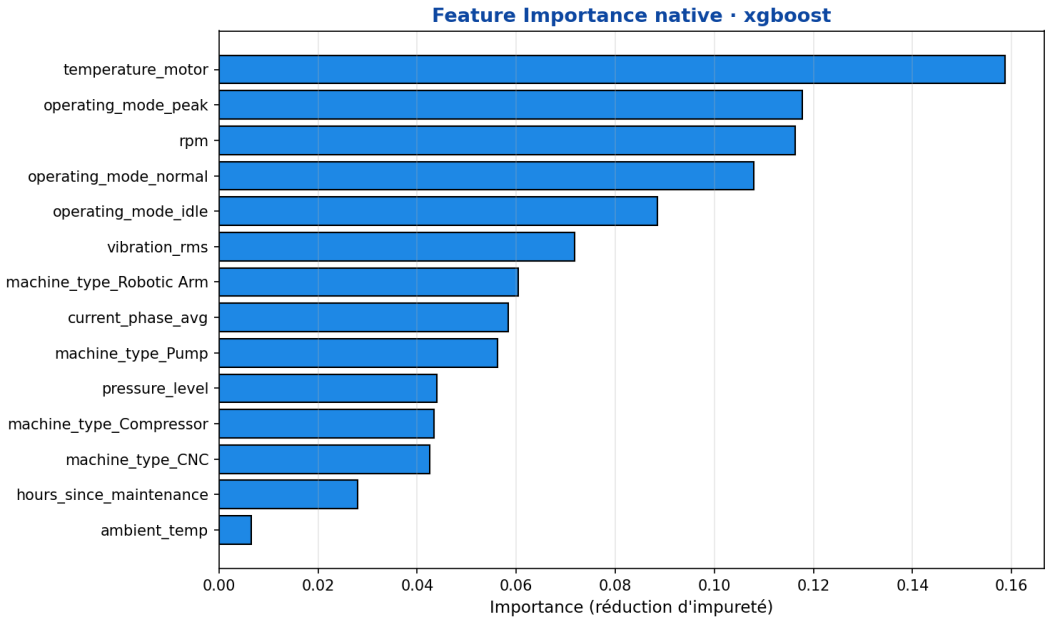


Figure 10 · Feature importance native (XGBoost). Vibration et température dominent · cohérent avec les corrélations de l'EDA et avec l'intuition métier.

Niveau 2 · Permutation importance

Méthode **agnostique au modèle** · on entraîne le modèle, on mesure son F1, puis on **mélange aléatoirement** les valeurs d'une feature et on remesure. Plus le F1 chute, plus la feature était importante. **Avantages** · marche sur n'importe quel modèle (même les MLP), pas biaisée. **Limite** · peut être instable sur petites données.

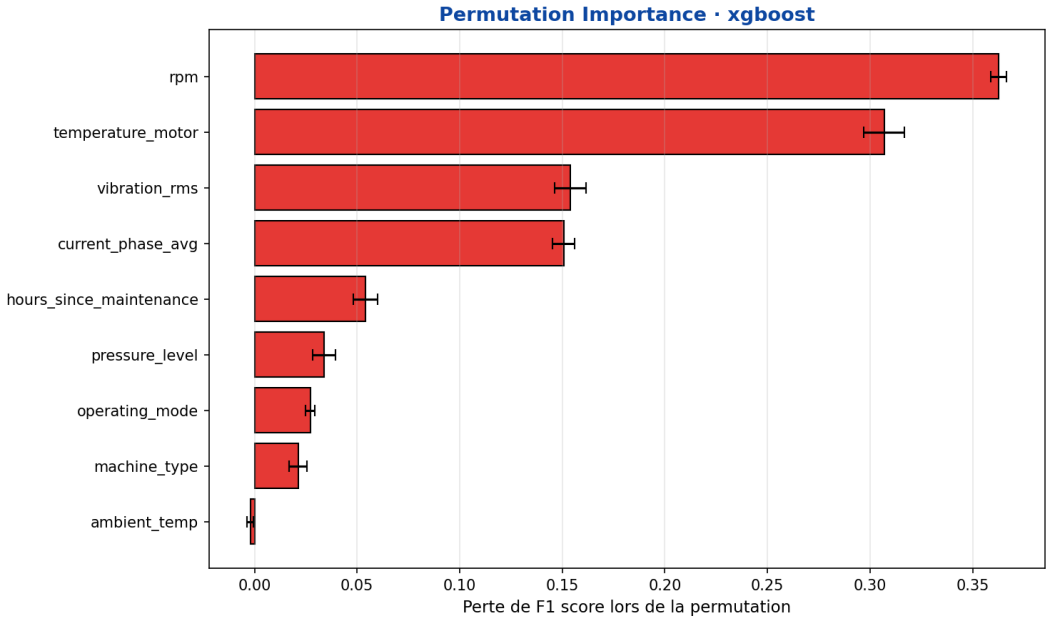


Figure 11 · Permutation importance (XGBoost). Les barres représentent la perte de F1 quand la feature est mélangée. Les barres d'erreur (±1σ sur 10 répétitions) indiquent la stabilité.

Niveau 3 · SHAP (SHapley Additive exPlanations)

Méthode théoriquement fondée (théorie des jeux, valeurs de Shapley). Pour CHAQUE prédiction individuelle, SHAP attribue à CHAQUE feature une contribution chiffrée (positive ou négative) qui s'additionne pour donner la prédiction finale. C'est l'outil de référence en explicabilité ML moderne.

DEUX VUES SHAP · LOCALE ET GLOBALE

Locale · pourquoi cette machine X précise a-t-elle une probabilité de panne de 0.82 ? → « parce que sa vibration est de 5.2 mm/s (+0.18 sur le score) et sa température de 78°C (+0.12) ». **Globale** · agrégation sur tout le test set → quelles features sont structurellement importantes pour le modèle ?

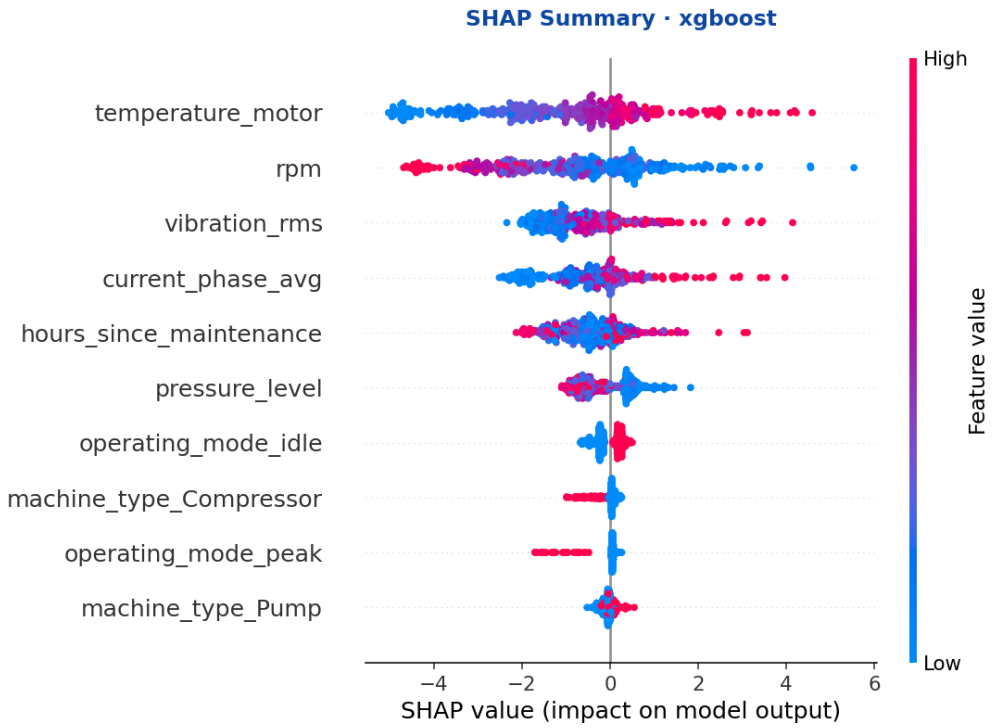


Figure 12 · SHAP Summary Plot. Chaque point = une observation du test set. Position horizontale = impact sur la prédiction (à droite = augmente la proba panne). Couleur = valeur de la feature (rouge = élevée, bleu = basse).

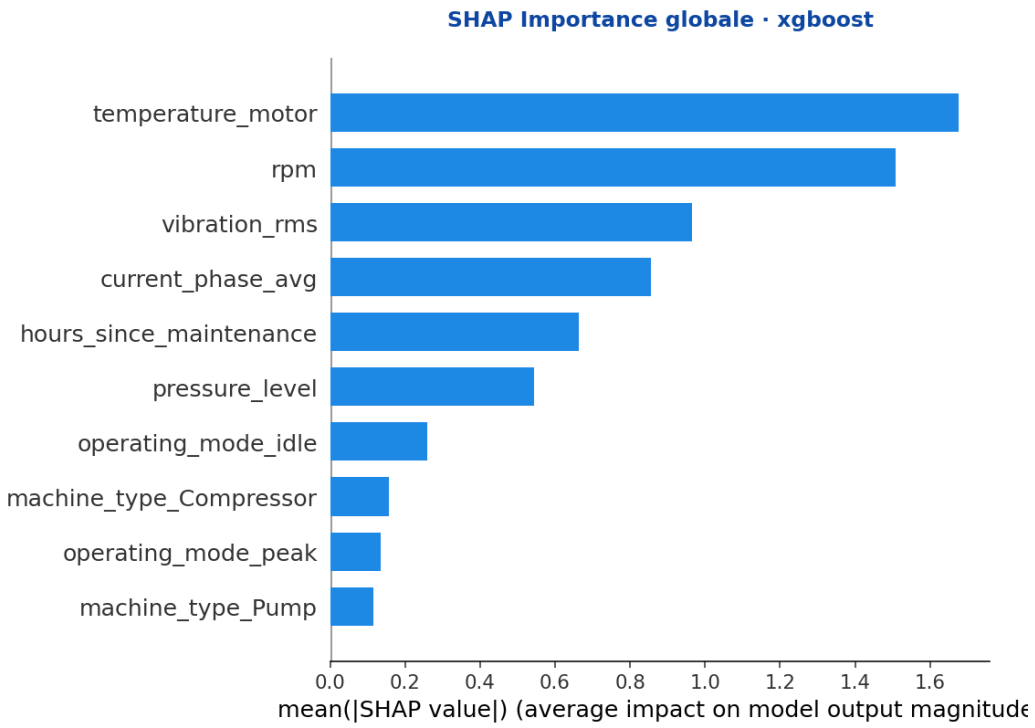


Figure 13 · SHAP importance globale · moyenne des |SHAP| par feature. Confirme que vibration et température sont les drivers principaux du modèle.

09

Mise en production · dashboard et API

Un modèle dans un notebook Jupyter ne sert à personne en entreprise. Il faut le rendre **utilisable** · soit via une interface graphique (dashboard) pour les utilisateurs non techniques, soit via une API pour les autres systèmes informatiques. Ce projet livre les deux.

Dashboard Streamlit · vision métier

dashboard/app.py cible le **responsable maintenance**, pas le data scientist. Conséquence · les KPIs techniques (F1, AUC, SHAP values brutes) sont relégués dans un onglet « Détails techniques » pour ne pas polluer l'écran principal du chef d'atelier. Cinq onglets ·

- **État du parc** · KPIs métier (machines critiques, coût à risque, RUL)
- **Plan d'intervention** · table d'actions priorisées par urgence
- **Impact économique** · ROI vs scénario réactif sans IA
- **Diagnostic machine** · saisie capteurs vers décision actionnable
- **Détails techniques** · EDA, modèles, SHAP (réservé DSI/jury)

API REST · industrialisation

api/main.py expose le modèle via FastAPI. Une API permet à d'autres systèmes (application mobile, ERP, SCADA) d'appeler le modèle par requêtes HTTP. Trois endpoints ·

POST /predict	Reçoit un JSON avec les valeurs capteurs, applique le pipeline, retourne la probabilité, la classe (0/1 selon le seuil métier) et le seuil utilisé.
GET /health	Vérification rapide que le service est vivant et que le modèle est chargé. Standard pour les load balancers et les outils de monitoring.
GET /model-info	Métadonnées · features attendues, version du modèle, métriques de performance, seuil de décision. Permet aux clients de s'auto-configurer.

POURQUOI UNE API + UN DASHBOARD ?

Le dashboard est l'interface utilisateur pour l'humain. L'API est l'interface pour les autres logiciels. En production, on veut souvent les deux · l'humain regarde le dashboard, le système SCADA appelle l'API toutes les heures pour scorer toutes les machines automatiquement. Architecture *Front → API → Modèle* standard.

10

Bonus · multi-classe, régression, calibration

Le sujet exige **une seule** tâche de prédiction. Ce projet en livre **trois** pour valoriser la grille de notation · la binaire principale + multi-classe + régression RUL.

Classification multi-classe · failure_type

Au-delà du panne oui/non, le script 07 prédit le **type** précis de panne parmi 5 classes (none, bearing, motor_overheat, hydraulic, electrical). XGBoost atteint **macro-F1 = 0.93**. **Intérêt métier** · le chef d'atelier peut mobiliser la *bonne équipe technique* avant d'arriver sur place · une équipe mécanique pour un problème de roulement, électrique pour un défaut de phase, etc.

POURQUOI MACRO-F1 ET PAS WEIGHTED-F1 ?

macro-F1 · moyenne simple des F1 par classe → chaque classe pèse pareil. **weighted-F1** · moyenne pondérée par fréquence → les classes rares pèsent moins. On préfère macro-F1 car on ne veut pas qu'une classe rare (ex. *electrical*) soit ignorée par le modèle juste parce qu'elle est minoritaire.

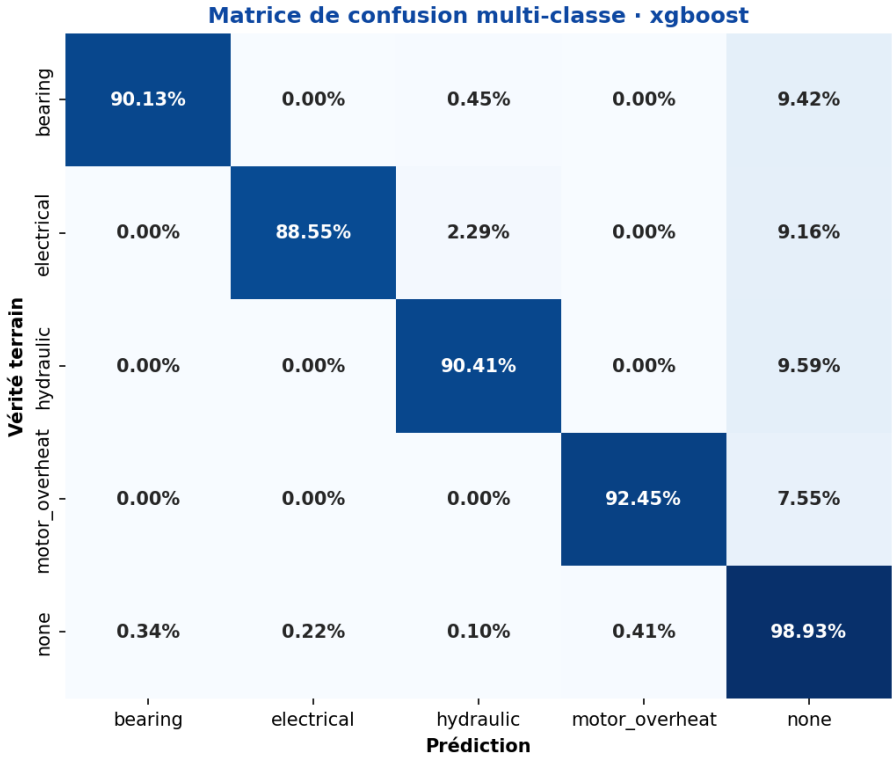


Figure 14 · Matrice de confusion multi-classe (XGBoost). Diagonale forte = bonne précision par classe. Les confusions résiduelles sont entre types proches.

Régression sur le RUL · Remaining Useful Life

Encore plus précis qu'une simple alerte oui/non · le script 08 prédit **combien d'heures** il reste avant la prochaine panne. Cible continue dans [0.5, 99] heures. Random Forest atteint **R^2 = 0.67** et **MAE = 9.6 h**.

MAE	Mean Absolute Error · erreur moyenne en valeur absolue. **9.6 heures** ici · « en moyenne, on se trompe de 9.6 heures sur la durée de vie restante ». Métrique très interprétable pour le métier.
RMSE	Root Mean Squared Error · pénalise plus fort les grosses erreurs. Utile si une erreur de 50h est bien pire qu'une erreur de 5h.
R^2	Pourcentage de variance expliquée. R^2=0.67 signifie « le modèle capture 67% de la variabilité réelle du RUL ». R^2=1 = parfait, R^2=0 = pas mieux que la moyenne, R^2<0 = pire que prédire la moyenne.

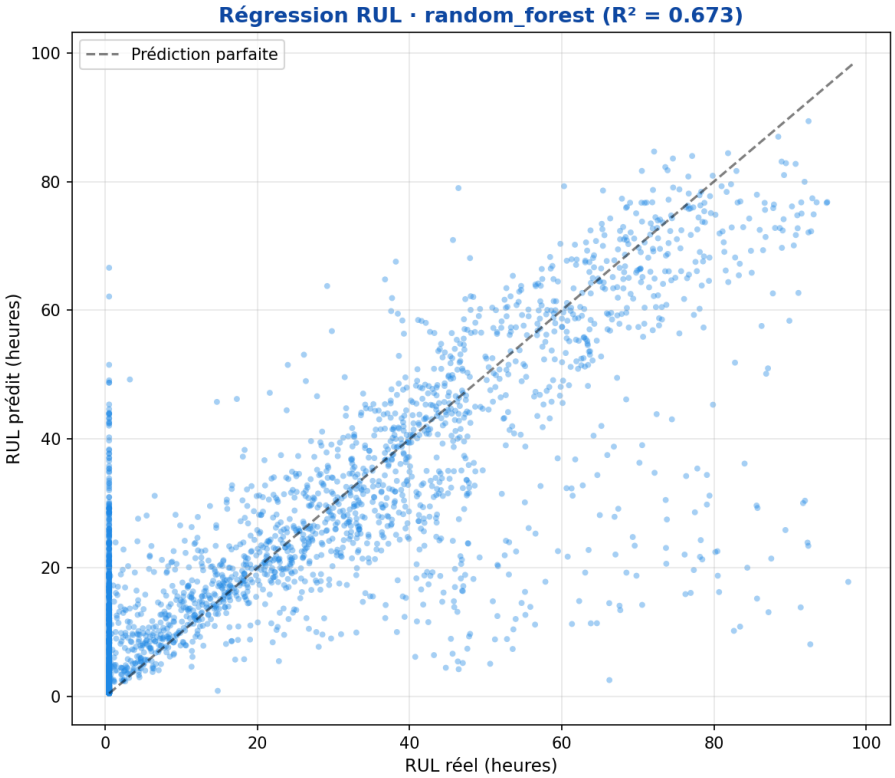


Figure 15 · Régression RUL · prédictions vs valeurs réelles. La diagonale = prédiction parfaite. Plus les points sont proches de la diagonale, mieux le modèle généralise.

Calibration probabiliste et seuil métier optimal

Quand le modèle dit « 0.73 », est-ce vraiment 73% de chance de panne ? Pas toujours · les modèles ML sont parfois **mal calibrés**. Le script 10 vérifie ça avec deux outils ·

Reliability diagram	On regroupe les prédictions par tranche de probabilité (ex. les 100 machines pour lesquelles le modèle a dit ~0.7). Si le modèle est bien calibré, ~70 doivent réellement tomber en panne. Plus la courbe colle à la diagonale, mieux c'est calibré.
Brier score	Mean Squared Error entre les probas prédites et les labels 0/1. **0 = parfait, 0.25 = aléatoire**. Notre modèle XGBoost atteint **0.025** · très bien calibré.

Seuil métier optimal · cost-sensitive thresholding

Par défaut, on classe « panne » si proba > 0.5. Mais ce seuil n'est pas optimal d'un point de vue économique. Avec coûts FN=1000€ et FP=100€ (ratio 10:1), on doit privilégier le recall (détecter les pannes) au prix de plus de fausses alertes. Le script scanne les seuils de 0.05 à 0.95 et trouve celui qui minimise **Coût = 1000 × FN + 100 × FP**.

RÉSULTAT · ÉCONOMIE 12 000 € PAR CYCLE

Seuil optimal trouvé · **0.23** (au lieu de 0.5). À ce seuil, le coût total est **32 600 €**. Au seuil 0.5 par défaut, c'est **44 600 €**. Économie · **–12 000 €** par cycle de scoring sur le test set. Ce seuil est consommé par l'API et le dashboard en production.

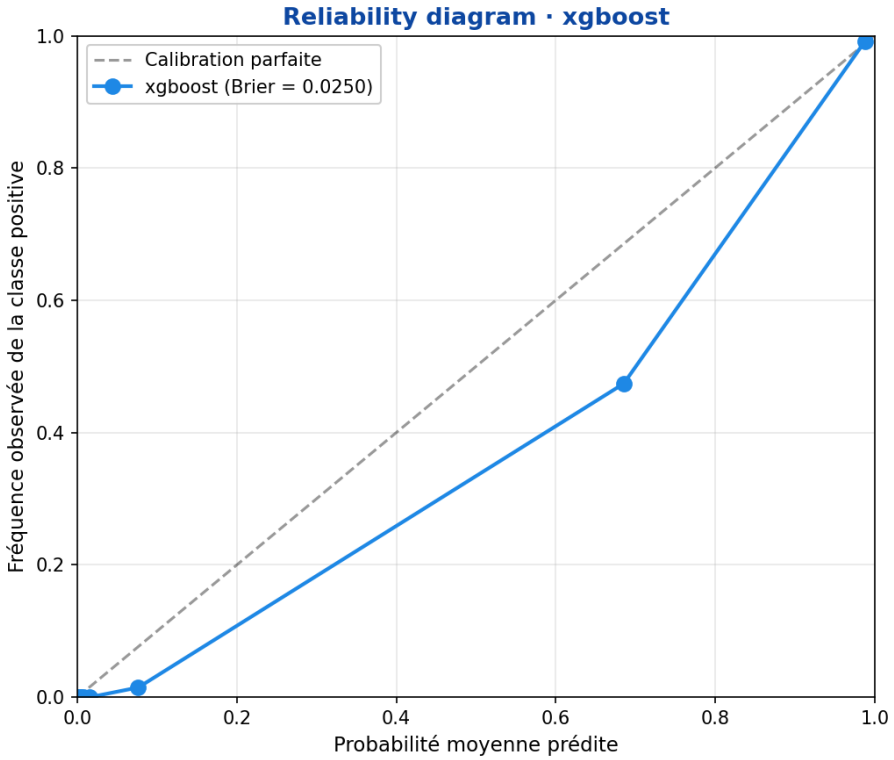


Figure 16 · Reliability diagram. La courbe colle bien à la diagonale → modèle bien calibré, on peut faire confiance aux probabilités.

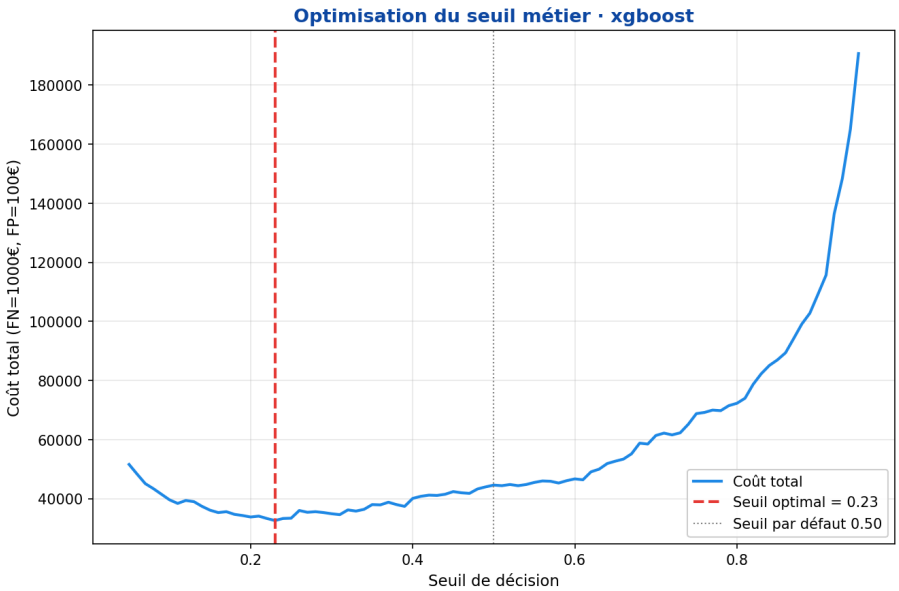


Figure 17 · Coût total en fonction du seuil. Le minimum à 0.23 est l'optimum métier.

Tuning d'hyperparamètres avec Optuna (TPE)

Les modèles ML ont des **hyperparamètres** · paramètres qui ne sont pas appris par l'algorithme mais choisis par le data scientist (nombre d'arbres, profondeur max, taux d'apprentissage, etc.). Bien les régler peut faire gagner plusieurs points de F1.

GridSearch	Teste toutes les combinaisons d'une grille définie à l'avance. Lent et exhaustif. 100×5×4 = 2000 essais.
RandomSearch	Tire des combinaisons au hasard dans des intervalles. Plus rapide mais pas plus malin.
Optuna (TPE)	Tree-structured Parzen Estimator · **apprend des essais précédents** pour proposer les hyperparamètres les plus prometteurs. Optimisation bayésienne · ~10× plus efficient que GridSearch.

Le script 09 lance Optuna sur 3 modèles (Random Forest, XGBoost, MLP), 20 trials × 3 CV folds par modèle = 180 entraînements. Meilleurs résultats trouvés ·

Modèle	Best F1	Best params
Random Forest	0.858	`n_estimators=250, max_depth=25, min_samples_leaf=2`
XGBoost	0.884	`n_estimators=500, lr=0.084, max_depth=10, subsample=0.75`
MLP	0.801	`architecture=128-64-32, alpha=0.002`

11

Synthèse, limites honnêtes et perspectives

Ce qui a été livré

- **4 modèles** ML/DL comparés rigoureusement (LR, RF, XGB, MLP)
- **3 tâches** de prédiction (binaire principale + multi-classe + régression RUL)
- **8 scripts** numérotés · pipeline reproductible end-to-end
- **33 figures** + 6 métriques par modèle + cross-validation 5-fold
- **Dashboard vision métier** · 5 onglets pour responsable maintenance
- **API REST FastAPI** · 3 endpoints + Swagger + tests httpx
- **Auto-install** des dépendances pip au lancement (zéro setup)

Limites assumées

Une démarche scientifique honnête expose ce qui ne marche pas ou ce qu'on n'a pas eu le temps de faire. Quatre limites majeures ·

Dataset simulé · les données Kaggle sont générées · les corrélations sont « trop propres » comparées à un parc industriel réel où les capteurs déraillent imprévisiblement, les machines sont hétérogènes en âge et en usage, et les modes de défaillance évoluent.

Pas de série temporelle · le split est aléatoire stratifié. Une vraie validation industrielle devrait tenir compte de l'ordre temporel (train sur passé, test sur futur) via une *sliding window cross-validation*. C'est plus dur et nous n'avons pas eu le temps de l'implémenter.

Seuil économique fixe · les coûts FN/FP sont supposés constants (1000€ et 100€). En réalité ils varient par machine (un robot critique coûte plus qu'un convoyeur secondaire), par criticité (haute saison vs basse saison), et par contexte (équipe disponible ou pas). Une version v2 permettrait des coûts dynamiques par machine.

Pas de drift monitoring · le modèle est entraîné une fois, déployé, et c'est tout. En production réelle, les distributions de capteurs dérivent (machines vieillissantes, remplacements, nouveaux modes opératoires) · il faut détecter ce *concept drift* et ré-entraîner périodiquement. Outils comme Evidently AI ou Whylogs ne sont pas intégrés ici.

Perspectives v2

- Intégrer du temps réel via Kafka + Spark Streaming sur les flux capteurs
- Déployer en MLOps · MLflow pour le tracking, Docker pour l'API, CI/CD GitHub Actions
- Drift detection automatique avec ré-entraînement déclenché
- Coûts métier dynamiques par machine · table de référence dans la BDD
- Dashboard temps réel · WebSocket pour les alertes push

ANNEXE

Conformité RNCP40875 Bloc 2

Le projet valide l'ensemble des compétences du Bloc 2 (BC2 · Piloter et implémenter des solutions d'IA en s'aidant notamment de l'IA générative). Le tableau ci-dessous donne pour chaque critère la preuve directe dans le repo.

Critère	Intitulé	Preuve
C3.1	Préparer/nettoyer les données	src/preprocessing.py · ColumnTransformer (Imputer + Scaler + OHE)
C3.2	Tableau de bord interactif inclusif	dashboard/app.py · Streamlit 5 onglets · Plotly · responsive
C3.3	Analyse exploratoire de données	scripts/02_eda.py · 8 figures + stats + analyse NaN
C4.1	Stratégie d'intégration de l'IA	Onglet Impact économique · ROI · ratio FN/FP 10:1
C4.2	Modèles prédictifs ML/DL	scripts/03 · 4 modèles dont MLP · F1=0.886 sur test
C4.3	Évaluation + écoresponsabilité	6 métriques + CV 5-fold + temps train/inference

Glossaire

EDA	Exploratory Data Analysis · l'étape d'observation des données avant modélisation.
Pipeline	Enchaînement reproductible de transformations · Imputer → Scaler → OHE → Modèle.
Data leakage	Quand l'info du test set fuite dans le train. Erreur classique du débutant.
CV	Cross-validation · découper le train en k parts pour évaluer la stabilité.
F1	Moyenne harmonique de Precision et Recall. Métrique principale en classification déséquilibrée.
ROC-AUC	Area Under ROC Curve. Mesure de qualité globale, peut être trompeuse en déséquilibre.
PR-AUC	Area Under Precision-Recall Curve. Plus dure, plus parlante en déséquilibre.
SHAP	SHapley Additive exPlanations · attribue une contribution chiffrée à chaque feature.
RUL	Remaining Useful Life · durée de vie restante d'une machine en heures.
Optuna TPE	Tree-structured Parzen Estimator · sampler bayésien efficient.
Calibration	Qualité des probabilités · un modèle calibré dit 0.7 → 70% de chance réelle.

MLOps	Machine Learning Operations · industrialisation du cycle de vie ML (data → model → prod).

Reproductibilité

Seed unique **RANDOM_STATE=42** propagé partout · auto-install des dépendances pip via *src/bootstrap.py* · chemins absolus calculés depuis *__file__* (portable Win/Linux/macOS) · 23 tests pytest · versioning sémantique. Le clone du repo + téléchargement Kaggle suffit pour relancer toute la chaîne.

Auteurs et contact

Adam Beloucif (20220055) · adam.beloucif@efrei.net
Emilien Morice (20241824) · M1 Mastère Data Engineering & IA
EFREI Paris Panthéon-Assas Université · Année universitaire 2025-2026
Repo · github.com/Adam-Blf/maintenance-predictive-industrielle